can have an incremental update mechanism that only surfaces new posts when they are published – but that’s another story, slightly off the point).

Now consider this caching mechanism for a billion walls: even if we store just the top 50 posts per wall, and for each post we just store its ID (so typically a few bytes), we would still need a terabyte for the cache.

These examples show already how we can easily reach a size, for the cache, that’s hard or impossible to keep in the RAM. It would still possible to use NoSQL databases, or distributed caches, like Memcached, Redis or Cassandra, to name a few, but even there, the more entries we’ll store, the slower it will be checking the cache: this will be clearer when we discuss the implementation of a typical cache.

And anyway, we can imagine a situation where we keep receiving different inputs, that would need new entries in the cache: as you can imagine, we can’t add new entries indefinitely to a cache: being a finite system, there will always be a point where an infinite number of entries will saturate it.

Therefore, at some point, once we have filled the cache, if we want to add new entries, we need to start purging some of the existing ones.

The question is, of course, which entries should we better purge? The somehow uncanny answer is that it would be ideal to remove the ones that won’t be requested anymore, or at least those that will be requested the fewer times.